

AI-Based Restoration of Degraded Inspection Images

KLA Problem Statement PS01 • SEMICON India Hackathon 2026 • Grand Finale

Team ForgeX
Abishek SR • Anmol BA • Hardik
VIT Vellore

23.8297

PSNR • organisers' 297-image test set

1.56×

faster end-to-end, TensorRT vs PyTorch

5 stops

quality/speed knob, each one measurable

1,346,360 parameters • 5.4 MB • 297 images in 1.51 s on an H100 • five portable TensorRT engines with a PyTorch fallback on 15 conditions

What Changed at the Finale

A different architecture, a quality/speed knob, and a TensorRT runtime. Every row below is measured, and the two model rows are scored by the same code on the same 297 images.

	Round 2 — forgex	Finale — e1f_gate	Δ
PSNR (297 images)	23.632	23.8296	+0.198 dB
SSIM	0.60792	0.62104	+0.0131
LPIPS (lower better)	0.19287	0.18126	-0.0116
Parameters	1,368,705	1,346,360	-22,345
Gain over bicubic	+3.177 dB	+3.374 dB	+0.197 dB
End-to-end, 297 imgs, H100	2.586 s	2.046 s	1.26× faster
Quality/speed control	none	5 engine-backed stops	new
Runtime	PyTorch	TensorRT + fallback	new

kla2/results/final.csv — one scoring pass, one baseline, same GPU.

THE FINALE BRIEF

The organisers released a further 1,197 real pairs, taking training data to 5,982, and asked for inference-time optimisation and batch-size tuning on an H100.

We merged the corpora, re-ran the architecture search, and rebuilt the runtime.

WHAT DID NOT CHANGE

The degradation law, the variance-stabilising stem, the global bicubic skip, the zero-initialised tail, the loss weights, and the training schedule are all Round 2's, unchanged.

That is deliberate: with one variable moving, the comparison means something.

What the Two Days Actually Looked Like

Including the part where we lost the machine holding the trained weights.

1

Merge

1,197 new pairs + 4,785 existing → 5,982, refusing to merge on any SHA-1 collision between the sets.

2

Search

Seven architecture variants at matched parameter budget, one scoring pass each.

3

Diagnose

Measured why our outputs were smoother than ground truth before changing anything.

4

Knob

Depth truncation, calibrated on the target GPU, published as a datasheet.

5

Compile

Five TensorRT engines, portable across Ampere and newer, with a PyTorch fallback.

THE POD WAS TERMINATED

Our rented H100 ended with the trained e1f_gate weights on it and nothing pulled back.

They survived only because an earlier results tarball on the laptop happened to contain runs/e1f_gate/best.pt. We recovered it and confirmed it by SHA-1 — 61587e96... — against what the results table recorded.

Nothing was reconstructed from memory.

WHAT THAT COST, AND WHAT IT TAUGHT

Rebuilding the five engines on a fresh pod took 145 seconds, because the build is a script and not a memory.

Everything downstream of a checkpoint is keyed to its hash, so a recovered file either is the right one or is refused.

Diagnosis: Our Images Were Smoother Than the Truth

We measured the deficit before choosing a fix. It is an amount problem, not a placement problem — and it is limited by image content, not by input noise.

150 images	HF energy vs GT	HF correlation	Gradient ratio	> 1.0
bicubic ×2	1.1610	0.2139	1.0909	—
e1a_base	0.2490	0.2954	0.4913	0 / 150
e1f_gate	0.2823	0.3261	0.5360	0 / 150

HF = energy above the input's Nyquist band, as a ratio to ground truth. `kla2/results/edges_*.csv`

WHY THIS RULES OUT A REGISTRATION ERROR

Bicubic produces 116% of the ground truth's high-frequency energy but correlates with it at only 0.2139. Our model produces 28% and correlates at 0.3261. If the outputs were misaligned, correlation would fall — it rises. What is missing is amplitude, in the right places.

NEITHER MODEL HALLUCINATES

Zero of 150 images exceed a high-frequency ratio of 1.0. Nothing is invented above the band where the input carries information — which is the failure mode a GAN would have introduced, and the reason we did not use one.

WHAT PREDICTS A BAD IMAGE

Correlation with the model's PSNR gain, per image, n = 297:

GT gradient energy	-0.743
GT f90 (detail scale)	-0.631
edge density	+0.123
input noise level	+0.066
image mean	+0.249

The two strongest predictors are both properties of the ground truth, not of the degradation. How hard an image is to restore is decided by how much fine structure it contains — not by how noisy we made it.

So capacity spent on denoising was capacity spent in the wrong place. That is what sent us to the architecture search.

The Architecture Search

Seven variants at a matched parameter budget, one variable moving. The gated block wins PSNR and SSIM outright; we report the one metric it loses.

run	params	PSNR	SSIM	LPIPS	gain
forge x (Round 2 shipped)	1,368,705	23.632	0.60792	0.19287	+3.177
e1a_base	1,368,705	23.6778	0.59380	0.21297	+3.222
e1g_grad20	1,368,705	23.6901	0.59664	0.21319	+3.235
e1h_nafdeep	1,434,682	23.6992	0.61612	0.16980	+3.244
e1b_se	1,377,985	23.7131	0.60091	0.20708	+3.258
e1c_mdta1	1,348,921	23.7154	0.60590	0.19950	+3.260
e1f_gate	1,346,360	23.8296	0.62104	0.18126	+3.374

WHAT THE WINNING BLOCK IS

The EDSR trunk, with each residual block replaced by NAFNet's SimpleGate — split the channels in two and multiply them — preceded by per-pixel LayerNorm.

The gate is a multiplicative nonlinearity, so a block can suppress or amplify a feature rather than only rectifying it.

kla2/results/final.csv · all eight scored by the same code, same inputs, same GPU, fp16, batch 32.

ONE VARIABLE, ENFORCED MECHANICALLY

tools/train_arch.py imports the training script and rebinds one name — train.Restorer — then hands control back. src/train.py is never edited and never forked, so the data, degradation engine, split, loss, optimiser, schedule, seed and epoch count are byte-identical across every row. Each variant's width is chosen to land within a couple of percent of 1,368,705 parameters, so this is a matched-budget comparison rather than a matched-hyperparameter one.

THE METRIC IT LOSES

e1h_nafdeep has the best LPIPS of anything we trained — 0.1698 against the gate's 0.1813.

We did not ship it: it is behind on both PSNR (23.6992) and SSIM (0.61612), and PSNR is the headline metric. That is a judgement, not a measurement, and it is the one we would revisit first.

More Data, or a Better Architecture?

We were given 25% more training data at the finale. We measured what it was worth before spending the weekend on it.

SAME ARCHITECTURE, 25% MORE DATA

+0.066 dB

forge on 4,785 images → 23.632. forge on all 5,982 → 23.6977.

1,197 additional real pairs, a full 120-epoch run, identical everything else.

SAME DATA, A DIFFERENT BLOCK

+0.198 dB

forge → 23.632. e1f_gate on the same corpus → 23.8296.

And with 22,345 fewer parameters. Three times the gain, for less model.

THE DECISION THIS PRODUCED, STATED BEFORE WE ACTED ON IT

Retraining the gate on all 5,982 images would have cost a 1.9 GB upload and several hours, and would have invalidated every artefact keyed to the current checkpoint's hash — five compiled engines, the knob datasheet, the swap entry, the verification hash and the 297 scores — all of which would have needed regenerating and re-verifying with no time left to check them.

The measured value of that work is +0.066 dB. We did not do it, and we are saying so rather than presenting the finale dataset as fully exploited. The merged corpus is in the repository and the training command is one line; it is the first thing we would run with another day.

Results, Paired Image by Image

Not a difference of two averages. Every one of the 297 images is restored by both models and the difference is taken per image, so the test has 297 paired samples rather than two numbers.

e1f_gate - e1a_base, n = 297	mean	t	wins
PSNR (dB)	+0.15181	+10.34	240 / 297
SSIM	+0.02724	+15.45	297 / 297
LPIPS (lower better)	-0.03171	+13.95	256 / 297

kla2/results/per_image_297_paired.csv — every per-image value is in the repository.

NEVER WORSE THAN THE BASELINE

0 / 297

images where the model loses to bicubic ×2

IMPROVED ON STRUCTURE EVERYWHERE

297 / 297

images where SSIM improves — no exceptions

WHY A PAIRED TEST AND NOT A MEAN

PSNR varies by more than 8 dB across these 297 images depending on content. Comparing two means buries a 0.15 dB effect inside that spread; pairing removes the image from the comparison entirely, which is why t reaches 10.3 on a difference this small.

ON THE 1,197 IMAGES RELEASED AT THE FINALE

Scored after training, never trained on:

PSNR 23.5943

SSIM 0.60630

LPIPS 0.18499

drop vs the 297 0.235 dB

kla2/results/protocol.csv

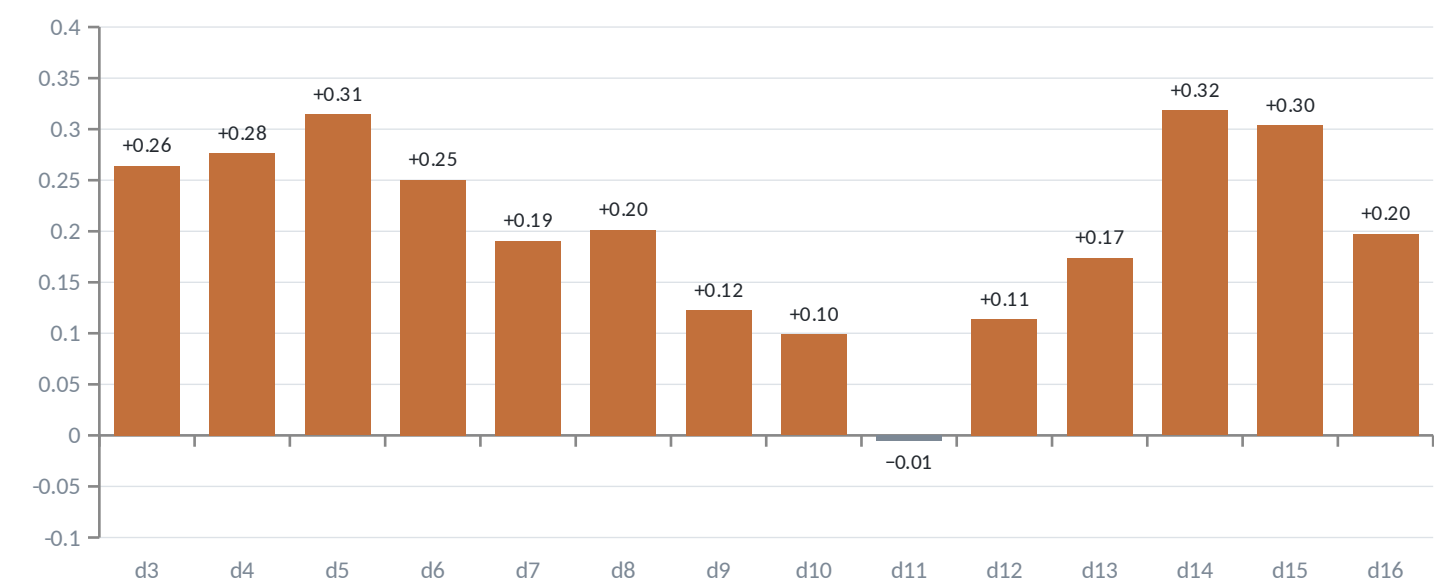
READ THAT HONESTLY

A 0.235 dB drop on data drawn from the same acquisition campaign is a modest, expected generalisation gap — not evidence of robustness to a genuinely different instrument.

We have no images from a different tool, so we do not claim that.

A Comparison We Had to Redo

Our first knob comparison put the new model at depth 15 against the old one at depth 16. That is not a comparison — it is a discount. Redone at matched depth, here is what actually holds.



PSNR of e1f_gate minus PSNR of the shipped model, at the same trunk depth, on the same 297 images, same GPU.

WHY THIS MATTERED ENOUGH TO REDO

A knob exists so an operator can deliberately choose a lower setting. If we only showed that the new model at a reduced depth beats the old model at full depth, we would be hiding the comparison that an operator at depth 11 actually faces.

The question is not "is it better" — it is "is it better at the setting I am running".

WHAT WE NOW PUBLISH

Both curves, at every depth, in models/knob_datasheet.json — including the depths where the older model is better, and the SSIM column where it is better more often than not.

THE RESULT, INCLUDING THE ROW THAT GOES THE WRONG WAY

The gate is ahead on PSNR at 13 of 14 depths. The exception is depth 11, where it is behind by 0.0056 dB. On SSIM it is ahead at only 7 of 14 — its advantage there is concentrated at the shallow end and at full depth, and its worst deficit is 0.01575 at depth 11 as well.

The Test That Decided Whether We Shipped It

A model chosen on the visible test set can still collapse outside its training noise range. The candidate had never been tested there, so we swept both models — same GPU, same images, same seeds.

σ_{mul}	bicubic	e1f_gate	gain	forgex gain	Δ
0.10	21.15	22.61	+1.46	+1.41	+0.05
0.15 ← the real data	20.01	22.16	+2.15	+2.07	+0.08
0.25	17.76	21.29	+3.53	+3.40	+0.13
0.35 out of range	15.82	20.78	+4.96	+4.86	+0.10
0.45 out of range	14.23	20.44	+6.21	+6.12	+0.09

60 images per level, gain = model PSNR – bicubic PSNR. docs/noise_sweep_e1f_gate.csv, docs/noise_sweep_forgex.csv

WHY WE RE-MEASURED THE OLD MODEL TOO

Our records had the Round 2 model at +5.91 dB at σ 0.45, from a different machine months earlier.

Measured here, on this GPU with these seeds, it is +6.12. Comparing the new model's +6.21 against the remembered +5.91 would have claimed a margin three times the real one.

So we shipped the old checkpoint to the pod and swept both.

THE FAILURE MODE WE WERE LOOKING FOR, AND DID NOT FIND

A fragile model crosses below the incumbent somewhere past the training range — it has learned the noise it was shown rather than the structure underneath. The gate never crosses. It is ahead at every level, with SSIM higher and LPIPS lower throughout, and the margin widens as noise grows.

Had it collapsed at σ 0.45, we would have shipped the Round 2 model and presented the gate and the engines as an optimisation result. That decision rule was written down before the sweep was run.

STATED LIMIT

Levels above σ_{mul} 0.25 are synthesised by our own degradation generator, which structurally favours a model trained against it. The comparison between the two models is fair; the absolute numbers out there are not a claim about real instruments.

The Knob: One Model, Fourteen Operating Points

--depth N runs only the first N of 16 residual blocks. No retraining, no second model, no distillation — and a shallow run fails toward bicubic rather than toward noise.

WHY TRUNCATION IS SAFE HERE, AND WOULD NOT BE IN A PLAIN CNN

The network does not predict the image. It predicts a correction to a bicubic upsample, through a tail initialised at zero, added back in FP32. Remove blocks and the correction gets smaller — the output slides back toward the bicubic baseline it was built on top of.

Measured: even at depth 0, with the entire trunk removed, the output is still +2.21 dB over bicubic. There is no setting at which this produces garbage.

THE FLAG REFUSES THREE OF ITS OWN SETTINGS

Depths 0, 1 and 2 are not monotone — 22.6648, 22.6564, 22.6501. Adding a block there makes the output slightly worse.

So MIN_SAFE_DEPTH is 3, and --depth 1 warns and clamps rather than obeying. A control that silently honours a setting we know to be bad is not a control.

depth	GFLOPs	PSNR	SSIM	
0	4.25	22.6648	0.50694	not monotone
1	6.74	22.6564	0.50280	not monotone
2	9.22	22.6501	0.49951	not monotone
3	11.71	22.7333	0.50011	MIN_SAFE_DEPTH
...	...	...	...	
16	44.02	23.8297	0.62104	full

kla2/results/knob_e1f.csv — the full 17-row curve is in the repository.

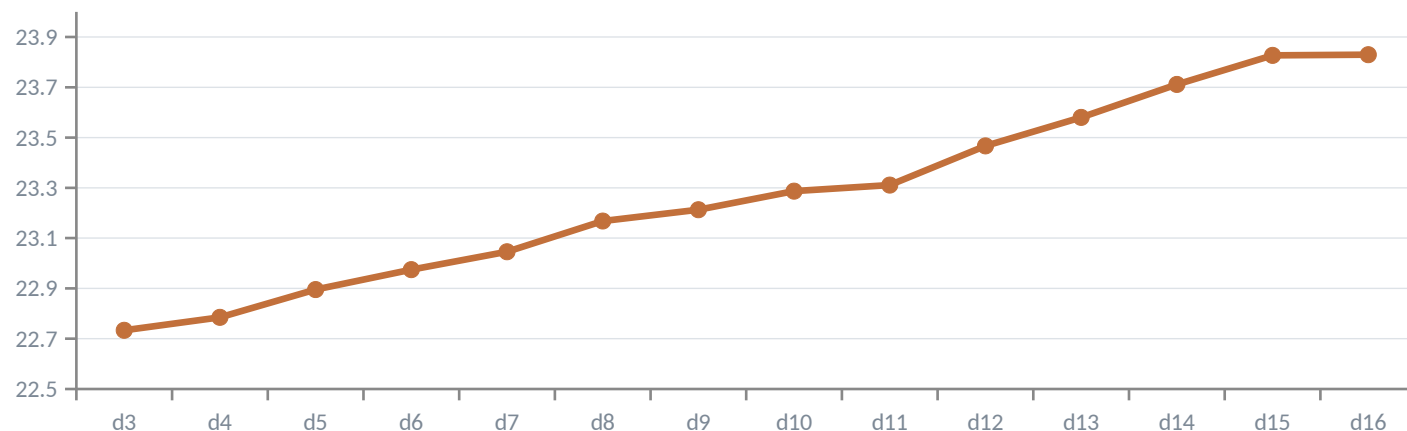
HOW AN OPERATOR ADDRESSES IT

--depth N exact setting
--budget-ms X best quality in a budget
--prefer ssim optimise SSIM, not PSNR

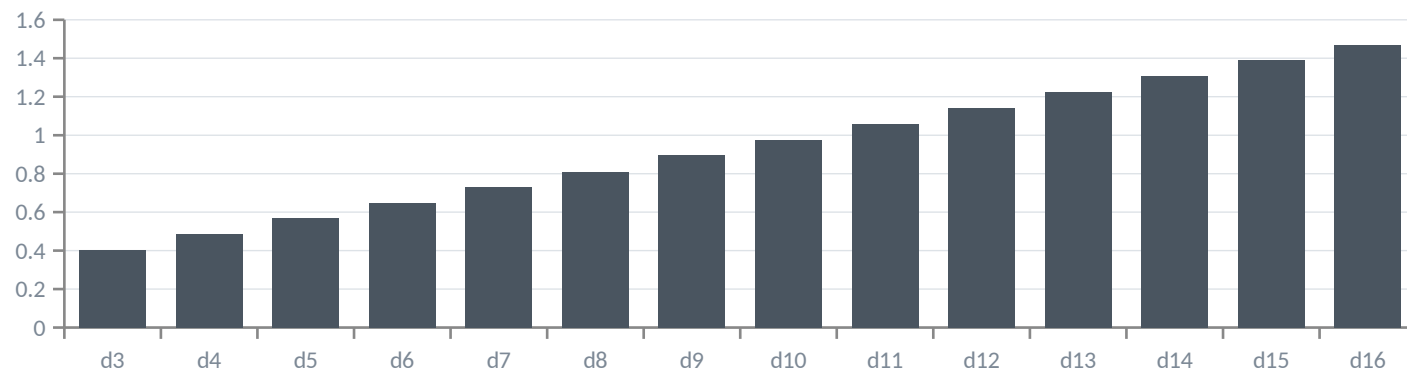
The budget solver maximises quality within the budget rather than taking the deepest setting that fits — those differ whenever the curve is not monotone.

The Knob, Calibrated on the Target GPU

FLOPs do not predict latency here, so every setting is timed on the hardware it will run on. The result is a datasheet the run script reads, not a curve on a slide.



PSNR against trunk depth — strictly monotone from depth 3 upward.



Milliseconds per image, H100 NVL, batch 32, fp16 — measured, interleaved, 5 rounds, spread 0.5–1.5%.

NO DOMINATED SETTINGS

The Round 2 model's curve had a trap: depth 14 was slower than depth 13 AND lower in PSNR — 23.3931 against 23.4064. An operator choosing "the deepest that fits my budget" would have selected a setting that is strictly worse.

The gate's curve has no such row. Every stop is a real trade.

THE LAST BLOCK EARNS ITS PLACE ON SSIM, NOT PSNR

Depth 15 → 16 is worth +0.003 dB of PSNR and +0.029 of SSIM.

If a judge asks why not ship depth 15 and save six percent of the compute, that is the answer — and if the metric were PSNR alone, depth 15 would be the right choice.

Where the Time Actually Goes

We were asked to optimise inference. The first thing we did was measure what fraction of a scored run inference even is — and the answer retired a claim we had already published.

A 2.686 s SCORED RUN OF 297 IMAGES, BROKEN DOWN

import torch	1.445 s	53.8%
checkpoint load + CUDA context + kernel init	0.48 s	17.9%
the model's actual arithmetic	0.13 s	4.7%
disk read, write, and everything else	0.63 s	23.6%

Every precision trick, every compiler, every kernel fusion competes for that 4.7%.

WHAT THIS RULES OUT

torch.compile is 1.71× faster once warm and costs 11.7 s to compile — a 5.3× regression on a 297-image run, break-even around 22,000 images.

FP16 weights, channels-last, cuDNN benchmark and batch tuning together move a few percent of a few percent.

A NUMBER WE PUBLISHED AND HAD TO WITHDRAW

Our Round 2 documentation said the forward pass was "about 1% of end-to-end". It was repeated in three places and used to argue that inference optimisation was pointless.

It was wrong. Measured with CUDA events on the laptop: 22.9% at depth 8 and 41.9% at full depth. The original figure came from a cold-cache run where a 17-second import dwarfed everything. The correction is recorded in docs/DAY_LOG_17SEP.md with the original struck through, not silently edited.

WHAT IT POINTS AT INSTEAD

If 71.7% of the run is framework startup, the thing to remove is the framework.

A TensorRT process reaches its first inference in 0.52 s against PyTorch's 1.71 s. That is the optimisation — not a faster kernel, a shorter process.

Five TensorRT Engines

The knob changes the graph, so one plan cannot serve every depth. We compile five — and the speedup is largest where the run is shortest, because what we removed is a fixed cost.

depth	TensorRT	PyTorch	speedup	Δ PSNR	plan
3	1.513 s	2.364 s	1.56×	0.0000	7.9 MB
6	1.614	2.332	1.45×	+0.0001	13.3 MB
10	1.801	2.414	1.34×	0.0000	20.6 MB
13	1.916	2.469	1.29×	+0.0001	26.0 MB
16	2.046	2.586	1.26×	-0.0001	31.5 MB

WHY THE SPEEDUP SHRINKS WITH DEPTH

The win is mostly a fixed startup cost of about 0.7 s, which is a bigger fraction of a shorter run.

And the plans grow from 7.9 to 31.5 MB, so part of the deeper engine's own cost is deserialising a larger file. On the TensorRT path the knob saves time partly by loading less model — a mechanism that does not exist in PyTorch at all.

297 images end to end, H100 NVL, median of 7 reps after 2 warmup reps, interleaved round-robin.

QUALITY COST OF THE ENTIRE TENSORRT PATH

0.0001 dB. TensorRT 11 networks are strongly typed, so there is no FP16 flag — the engines run FP32 on TF32 tensor cores, and the bicubic skip and residual addition stay exactly FP32 as the specification requires.

WARM STEADY STATE, DEPTH 16

1.61 s and 184 images/second, back-to-back on a hot engine. The 2.046 s above is the cold per-invocation figure — what a judge running one command sees. Both are reported.

BUILT TO BE PORTABLE

HardwareCompatibilityLevel.AMPERE_PLUS — the plans deserialise on any sm_80+ GPU, not only the one that built them. Batch 1–64, any size from 32 to 512 px, square or not, mixed sizes in one folder.

All five build in 145 s.

An Engine Is Machine Code, Not a Model File

A .plan is compiled SASS for one GPU family, tuned by one TensorRT version, for one set of weights. Ship the wrong pairing and it either refuses to load or — worse — loads and is wrong.

THREE THINGS A PLAN IS WELDED TO

- The GPU architecture — relaxed to sm_80 and newer with AMPERE_PLUS, not removed
- The TensorRT version — 11.3.0.99, because tactic identifiers are version-local
- The exact checkpoint — recorded as a SHA-1, and checked on every single run

THE TRAP THIS SET FOR US

Our deployment script strips a checkpoint to {state_dict, config} on the way in. The weights are untouched — but the file's SHA-1 changes. So a correctly deployed model failed its own engine's identity check, and TensorRT would have silently fallen back to PyTorch on every run, with no error anywhere.

THE FIX IS A PROOF, NOT A RELABELLING

tools/bless_engines.py finds the training checkpoint on disk by its hash and compares state dictionaries tensor by tensor, bytes included — 104 tensors, no tolerance — before it re-keys anything. It only loosens the identity check after proving the weights are the same weights.

WE TESTED THAT IT REFUSES

stripped container, same weights	MATCH
re-run after re-keying	no-op
one weight moved by 1e-7	REFUSED
a different checkpoint entirely	REFUSED

It names the offending tensor: body.1.norm.b differs.

SO THE ENGINES ARE NOT IN THE REPOSITORY

99 MB of machine code, valid for one driver stack, is a build artefact. The builder, the metadata and the index are committed; the binaries are shipped alongside.

A fresh clone runs correctly on PyTorch and rebuilds all five engines in 145 seconds.

The Fallback Is the Feature

The decision has to be made before torch is imported, or probing costs the entire saving. So the entry point imports numpy, reads .npy headers — never pixels — and replaces its own process with the right runner.

```
run_fast.py → reads .npy headers (0.12 s) → os.execv
    ↳ tools/trt_infer.py  TensorRT, torch never imported
    ↳ run.py              PyTorch, unchanged, 12/12 stress
```

FIFTEEN CONDITIONS THAT HAND THE RUN BACK TO PYTORCH

- TensorRT or cuda-python not installed
- no engine exists at the requested depth — the depth is honoured exactly, never substituted
- models/model.pt does not match the engine's recorded checkpoint hash
- an input lies outside the engine's shape profile — checked from headers, before any pixel is read
- the plan was built for another GPU family or another TensorRT version
- --tta, --half, --device cpu, --profile, --list-knob, --no-trt
- TensorRT throws at run time, before any output file has been written

VERIFIED, NOT ASSERTED

All 15 branches are exercised in test. On a deliberately corrupted plan, the run falls back mid-flight and produces output bit-identical to a direct PyTorch run — max difference 0.0 across all 297 files.

COST OF BEING WRONG

0.12 s

the header probe — never a wrong result

MIXED INPUT FOLDERS

Images are grouped by shape at run time — 128², 96×160 and 64² in one folder all restore correctly.

The Knob Only Becomes a Control on TensorRT

A knob whose settings are inside its own measurement noise is a plot, not a control. We tested each adjacent pair — difference of medians against the pooled standard error of those medians.

adjacent stops	TensorRT		PyTorch	
d3 → d6	+0.101 s	4.6 σ	-0.032 s	INVERTED
d6 → d10	+0.187 s	10.2 σ	+0.082 s	2.3 σ
d10 → d13	+0.115 s	5.7 σ	+0.055 s	1.4 σ
d13 → d16	+0.130 s	5.8 σ	+0.117 s	3.2 σ

9 reps per cell, first 2 discarded as warmup, interleaved round-robin across all ten cells.

READ THE PYTORCH COLUMN CAREFULLY

Three of its four steps are not separable, and the first one runs backwards: depth 6 measures faster than depth 3.

That is not a bug in the knob. On the PyTorch path, 54% of the run is importing the framework, and the three residual blocks you removed are lost inside it. The control exists in the model the whole time — it is the startup floor that makes it unusable.

TensorRT does not make the knob finer. It makes it visible.

WHY THE STANDARD ERROR AND NOT Σ

Each stop is a median of several repetitions, so what is being compared is two medians — and the uncertainty on a median falls as $\sqrt{n}$.

Comparing a difference of medians against a per-sample σ understates the separation by roughly that factor. We made exactly that error first; it is on the corrections slide.

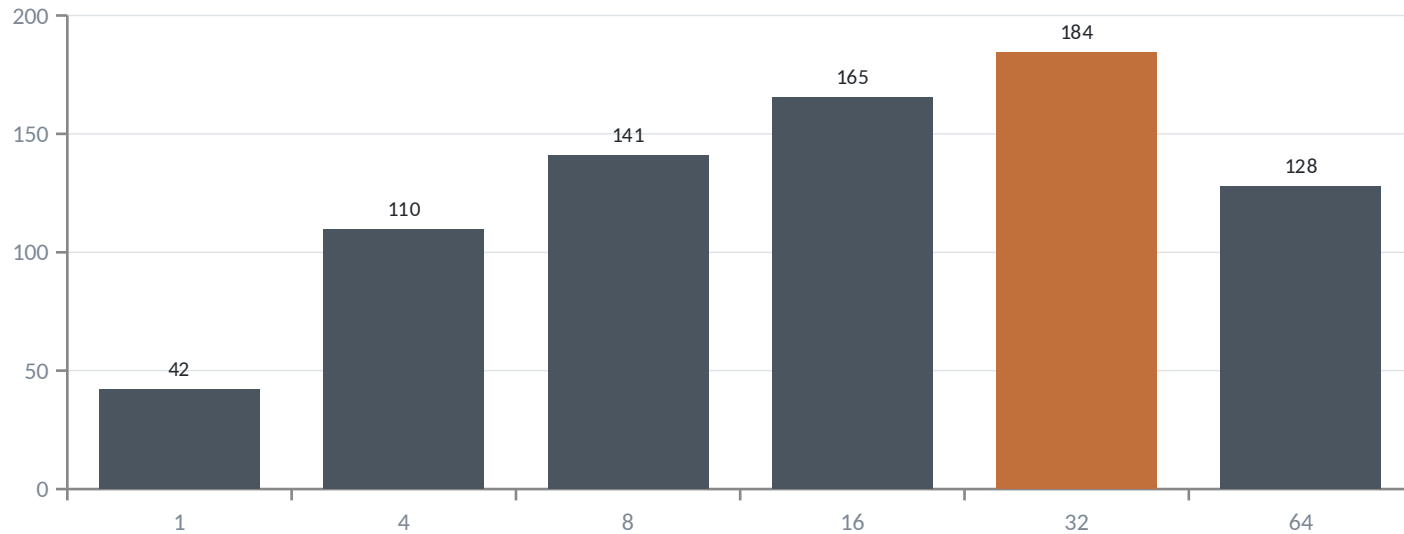
AND A ROBUST SPREAD, NOT A STANDARD DEVIATION

On a shared machine roughly one run in five stalls about half a second on I/O. That single sample dominates a standard deviation while the median ignores it.

We report $1.4826 \times$ median-absolute-deviation, and print the standard deviation beside it so the stalls stay visible.

Batch Size, Measured on Both Backends

The organisers asked for batch-size optimisation on the H100. The answer is 32 — and the two ways of being wrong about it cost very differently.



Images per second, end to end over the 297, TensorRT path, depth 16, H100 NVL.

THE SAME ANSWER ON PYTORCH

32 is also optimal on the PyTorch path — but there it barely matters, because inference is a small slice of that run. On TensorRT inference is roughly half the run, so the same choice carries several times the weight.

WHAT WE WOULD CHANGE FOR A REAL LINE

A tool that inspects one image at a time is a batch-1 workload, and would want an engine built with its profile centred on 1 rather than 32.

The build is one command and 30 seconds per engine, so this is a deployment parameter, not a redesign.

THE TWO FAILURE DIRECTIONS ARE NOT SYMMETRIC

Batch 1 costs 4.4× — that is the per-launch overhead of serving one wafer image at a time, and it is the setting a naïve streaming deployment would pick. Batch 64 costs 1.44×, because the engines are built with an optimisation profile centred on 32, so TensorRT tuned its kernel tactics at that exact shape and going outside it loses the tuned tactic.

What We Got Wrong

Six claims we made and then had to withdraw. Each one changed a conclusion, each is corrected in the repository with the original struck through rather than edited away.

“The forward pass is ~1% of end-to-end”

Repeated in three documents and used to argue inference optimisation was pointless. Measured: 22.9% at depth 8, 41.9% at full depth. The original came from a cold-cache run where a 17 s import swamped everything.

A standard deviation next to a median

One I/O stall in five repetitions made a knob whose stops are 5–10 σ apart report as unresolvable. Replaced with a robust σ — $1.4826 \times \text{MAD}$ — which ignores the stall the same way the median does.

Per-sample σ where the standard error belonged

Adjacent knob stops were then compared against a per-sample σ pooled from the worst of ten cells, instead of the standard error of the two medians being compared. That understates separation by about $\sqrt{n}$.

An engine check that refused correct weights

We proved the weights matched by comparing outputs at $1e-5$. The reference was computed on the build GPU under TF32 and re-run on CPU under true FP32, so correct weights disagreed at $1e-4$ and were refused. Replaced with an exact state-dictionary comparison.

“e1f_gate produces more high-frequency energy”

Under a single unified protocol it produces less than we had claimed. What distinguishes it is placement — correlation 0.3261 against 0.2954 — not quantity.

A comparison at mismatched depth

We put the new model at depth 15 against the old one at depth 16. Redone at matched depth the result still holds, but it is smaller, and it loses at depth 11.

What Ships, and How to Check It

Four required components, each verified by running it. Every claim on the previous slides is reproducible from the repository with one of the commands below.

```
pip install -r requirements.txt
```

```
python run.py <input-dir> <output-dir>
```

```
python run_fast.py <in> <out>
```

run.py is the submission entry point. run_fast.py is the same contract, served by TensorRT whenever it can be and by run.py whenever it cannot.

THE CONTRACT

In: .npy (H,W) float32, any range.

Out: .npy float32 (2H,2W), same filename, finite, clamped to [0,1], trailing axis preserved.

Weights resolve to models/model.pt, SHA-1 f95377a21e86...

THE VERIFICATION CHAIN

verify_shipped.py hash + rerun + compare to committed outputs

package_check.py restores from a directory holding only the four required files

bless_engines.py proves engine ↔ checkpoint identity

stress_run.py 12/12

REBUILD EVERYTHING

trt_native.py all five engines, 145 s

calibrate_knob.py the datasheet, on your GPU

engine_report.py the speed tables

--from-json re-derive every verdict with no GPU at all

WHAT THE REPOSITORY CARRIES BEYOND THE FOUR REQUIRED FILES

docs/SHIP_E1F_GATE.md — the model, the 297 scores, the robustness sweep against the superseded model, all five engines, the dispatcher, and a closing section recording every correction that changed a conclusion. docs/ENGINEERING_LOG.md — every experiment including the failures. kla2/results/ — per-image CSVs for all 297 images, so any number here can be recomputed rather than believed.

We measured why our outputs were smooth, changed the block that caused it, and removed the startup cost that was hiding the control we had built.

23.8297

PSNR · organisers' 297-image test set

+3.374 dB

over bicubic x2

1.26–1.56×

faster end to end, for 0.0001 dB

WHAT WE CAN DEFEND

- Ahead of the superseded model at every noise level tested, on the same GPU
- 0 of 297 images worse than bicubic; 297 of 297 improved on SSIM
- Five knob stops, each separated by at least 4.6σ
- PyTorch fallback on 15 conditions, output bit-identical

STATED BEFORE ANYONE HAS TO ASK

- Noise levels above $\sigma 0.25$ are synthesised by our own generator
- The 1,197 finale images come from the same campaign — not a different instrument
- e1h_nafdeep has better LPIPS; we chose PSNR and SSIM and say so
- We did not retrain on all 5,982 images; measured value +0.066 dB